

SISTEMA DE RECOMENDAÇÃO

Prevendo o próximo carrinho com MLP + Embeddings — End-to-End

PyTorch

Scikit-Learn

DVC

MLflow

Docker

M L P M A R K E T R E C O M M E N D E R

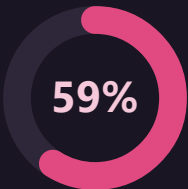
SITUAÇÃO · STAR : S

O Problema



Contexto

- E-commerce de supermercado quer aumentar a **recorrência** prevendo o que cada cliente compra no **próximo pedido**
- Não existem notas nem avaliações — só **feedback implícito**: o usuário comprou ou recomprou



dos **33,8 milhões de itens** já haviam sido comprados antes pelo mesmo usuário — **recompra é o sinal dominante** do domínio

Escala do dataset (Instacart)

3,4 mi
pedidos reais de supermercado

206 mil
usuários, 4 a 100 pedidos cada

50 mil
produtos em 134 corredores



TAREFA · STAR : T

Nossa Solução

- 1 Histórico de pedidos do usuário (`prior`)
- 2 Janelas temporais por usuário — sem vazamento de futuro
- 3 Geração de candidatos — 4 fontes, até 200 produtos/usuário
- 4 17 features temporais + 4 embeddings
- 5 Modelo final: MLP ranker PyTorch [128 → 64 → 1]
- 6 Top-K por usuário → MLflow Registry `@production`

Metas

NDCG@10

> **melhor baseline**

Comparação

**4 métricas × 4
baselines**

Reprodutibilidade

dvc repro exato

AÇÃO · STAR : A

Dataset Supervisionado — Janelas Temporais



92,7 mi

pares (janela, candidato)
3,7% positivos

3

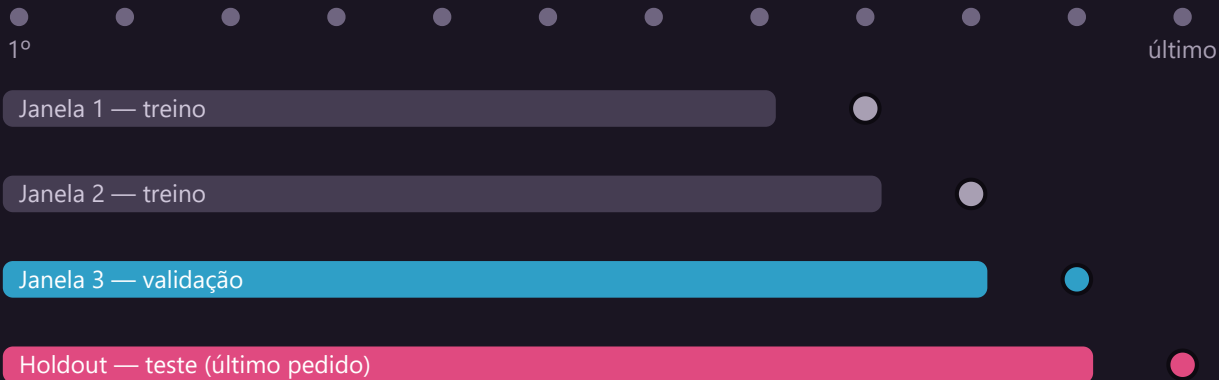
janelas por usuário
2 treino · 1 validação

115.909

janelas de teste
último pedido de cada usuário

Um usuário elegível — barra = histórico da janela · ponto = pedido-alvo

pedidos do usuário —



△ Anti-leakage: cada janela tem corte temporal próprio — features usam **apenas** eventos anteriores ao pedido-alvo; scalar e mapas ajustados só no treino

AÇÃO · STAR : A

Dois Estágios — Geração de Candidatos



O recall máximo do sistema é definido aqui: candidato que não entra na lista nunca será recomendado

AÇÃO · STAR : A

O que o Modelo Enxerga



 Recompra & recência (par usuário-produto)

`purchase_count · reorder_count ·
orders/days_since_last_purchase ·
was_bought_before`
→ comprou há pouco e compra sempre = volta ao carrinho

 Comportamento do usuário

`reorder_rate · avg_basket_size ·
avg_days_between_orders · prior_order_count`
→ calibra quantos itens e com que cadência cada um compra

 Afinidade de categoria

`user_aisle_purchase_count ·
user_department_purchase_count`
→ generaliza para produtos pouco vistos via corredor/departamento

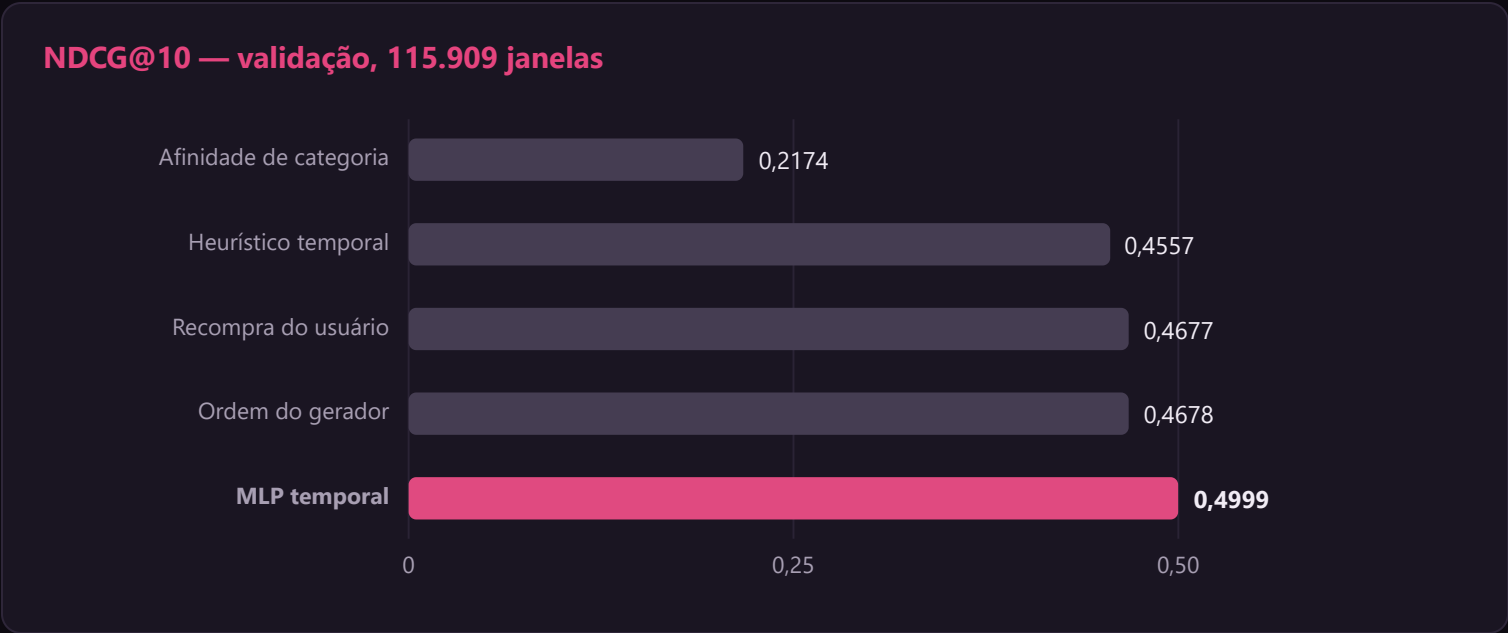
 4 Embeddings (dim 32)

`user_id · product_id · aisle_id · department_id`
→ afinidades latentes que features manuais não capturam; índice UNK para categorias não vistas

AÇÃO · STAR : A



Do Baseline ao Modelo Final



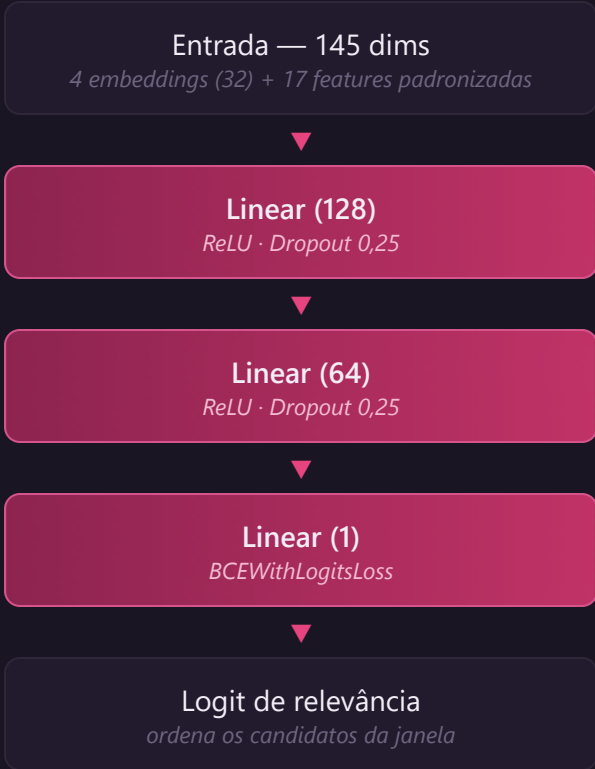
🏆 **Por que o MLP venceu:** ganho de **+0,032 absoluto** sobre o melhor baseline, consistente também em hit rate, recall local e precisão — num domínio onde a recompra torna os baselines quase imbatíveis.

AÇÃO · STAR : A

MLP — Arquitetura e Decisões



Arquitetura



Decisões técnicas

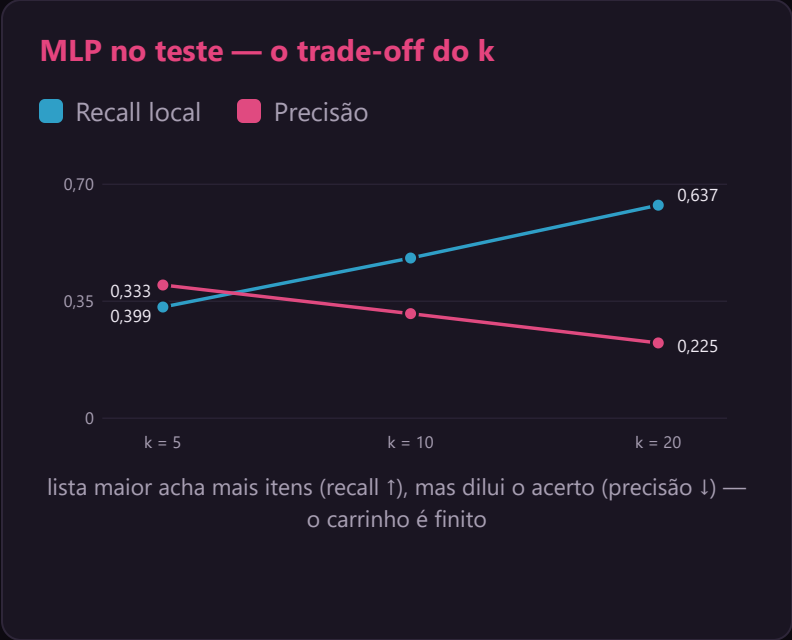
- **Embeddings** — 206 mil usuários e 50 mil produtos sem one-hot; UNK (`padding_idx=0`) cobre categorias novas
- **Adam** lr 1e-3 + **ReduceLROnPlateau** (patience 5) quando a validação estagna
- **Early stopping** por NDCG@10 — melhor época: **2** (depois, a loss cai mas o ranking piora: overfitting contido)
- **Batch 8192** — 92,7 mi de pares exigem throughput
- **Seed 42** em Python, NumPy e PyTorch

AÇÃO · STAR : A

Avaliar Ranking, não Classificação



- NDCG@k**
qualidade da **ordenação** — métrica primária (k=10)
- Hit rate@k**
pedidos com **≥ 1 acerto** no top-k
- Recall local@k**
dos positivos **cobertos pelos candidatos**, quantos chegam ao top-k
- Precisão@k**
fração do top-k **comprada de fato**



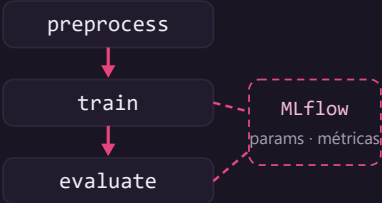
📌 **Recall global $\approx 0,34@10$** mede o sistema completo (geração + ranking): é o teto imposto pelo gerador de candidatos, não um defeito do MLP — melhorá-lo é trabalho de cobertura, não de modelo

AÇÃO · STAR : A

Da Pesquisa para Produção



Código & Pipeline



grafo real do pipeline (dvc dag)

- Notebooks → `src/` com SOLID e **Factory** de modelos
- Dados no GCS — `dvc pull` **sem credencial**
- Config via `.env` + Pydantic Settings

Infra

- Docker **multi-stage** — torch CPU no Linux, usuário não-root
- Compose: MLflow server + treino na mesma rede
- MLflow em produção no **Cloud Run** — URL pública, custo ~zero
- Registry: `@staging` → `@production`

Qualidade

38
testes passando

- ruff sem erros + **pre-commit**
- `uv.lock` — instalação idêntica em qualquer máquina
- Commits semânticos em equipe

Cada dvc repro loga params + métricas no MLflow · inferência via `scripts/predict.py` com fallback de popularidade

RESULTADO · STAR : R

O que Entregamos



Generalizou: NDCG@10 de **0,4948 no teste** (vs 0,4999 na validação) — em 10 sugestões, **91% dos pedidos têm acerto**

Verificável: MLflow público no Cloud Run — 3+ runs e modelo **@production** · bônus de deploy em nuvem



Conclusões e Próximos Passos

🧠 Aprendizados-chave

- **Baselines fortes são o teste de honestidade** — num domínio dominado por recompra, o ganho da rede é real e sabemos de onde vem
- **Sem leakage por desenho** — janelas temporais com corte próprio valem mais que qualquer correção posterior
- **Reprodutibilidade é decisão de projeto** — seeds, lock file, dados versionados e pipeline determinístico
- **Infra de custo zero tem trade-offs** — SQLite sobre GCS FUSE exigiu entender a semântica de escrita do bucket

✅ Entregues

- Model Card — métricas, limitações e vieses
- Reprodução do zero: `uv sync + dvc pull` sem login
- MLflow público com Registry `@production`

🚀 Próximo passo

- Ampliar cobertura do gerador de candidatos